

Laboratorio 1: Modelado, Identificación y Simulación de un Sistema de Control en Tiempo Discreto

Andres Camilo Bernal Ospina, 7003932. Filocarís Triana Pinzón, 7003936

Resumen—

Palabras clave—

I. INTRODUCCIÓN

II. DESARROLLO DE LA PRÁCTICA

Para el diseño y desarrollo del seguidor de línea se comenzó con la definición de la forma del chasis y los elementos que lo componen.

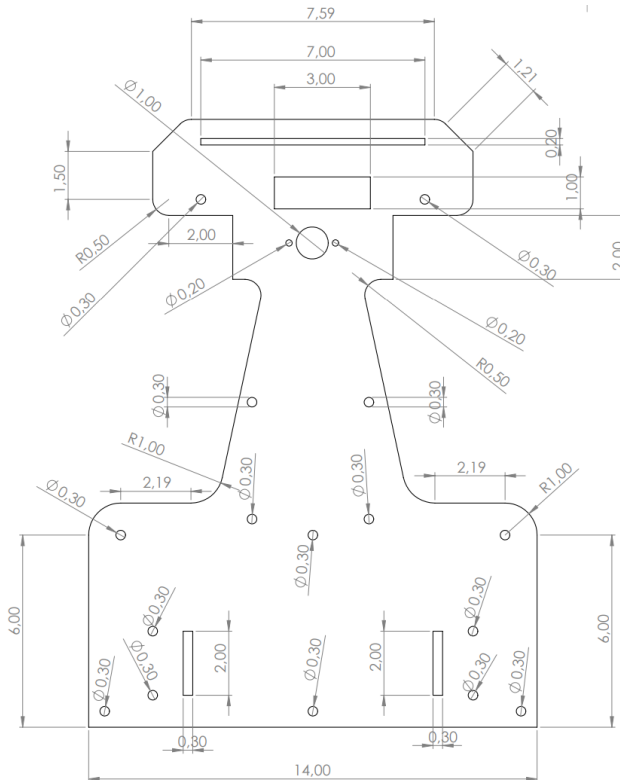


Figura 1. Plano con cotas del seguidor.

Como se aprecia en la figura 1, se definieron las medidas y el espacio del seguidor para poder ubicar todos los componentes que se requieren para la construcción del seguidor.

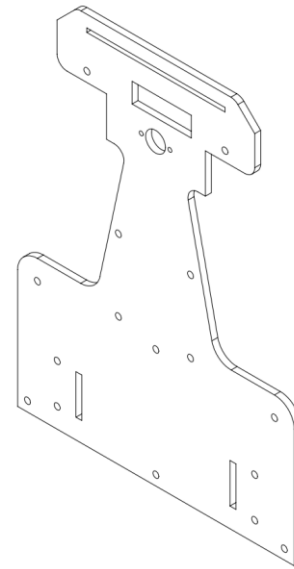


Figura 2. Vista Isométrica del chasis principal.

Se dispuso también de un segundo chasis elevado ubicado en la parte trasera del seguidor como se ve en la figura 3 y 4.

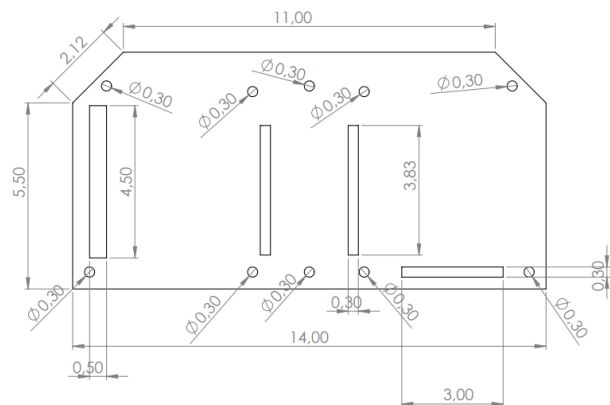


Figura 3. Segundo chasis.

Este segundo chasis se hizo con la finalidad de facilitar conexiones y asegurar que la mayor parte del peso se concentre en la parte trasera sobre el eje de las ruedas.

Esto también ayuda con el orden en la futura distribución de los elementos mecánicos y eléctricos que componen el seguidor de línea.

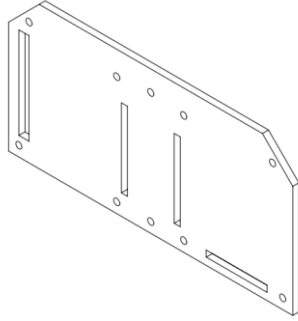


Figura 4. Vista isométrica del segundo chasis.

Componentes usados:

Tarjeta controladora: ESP 32.
Driver puente H: L298N.
Sensor seguidor digital: QTR 8 RC.
Regulador de Voltaje: LM2596.
Motores reductores 3000 RPM.
Mini Rueda loca 3PI.

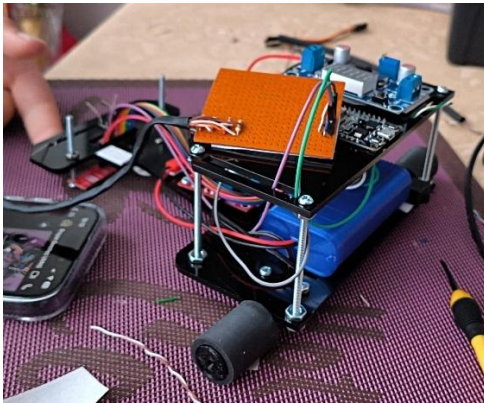


Figura 5. Montaje del seguidor de línea.

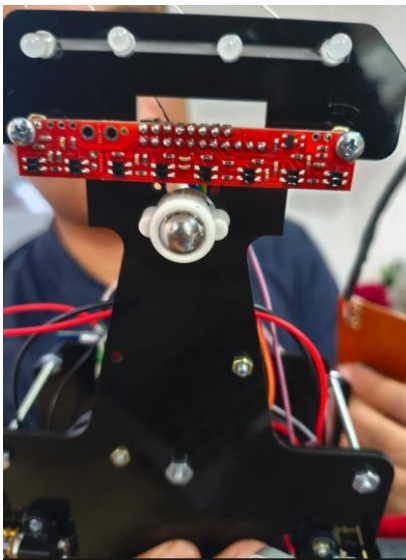


Figura 6. Vista inferior del seguidor.

Para comenzar con el modelo cinemático y el dinámico comenzamos definiendo los siguientes datos:

Geometría y masas (Cuerpo del seguidor):

R: radio de la rueda [m].
2L: distancia entre centros de ruedas (ancho de seguidor) [m].
d: offset del centro de masa (COM) respecto al punto medio del eje (positivo hacia adelante) [m].
m_r: masa del chasis sin ruedas ni motores [kg].
I_s: inercia de la plataforma alrededor del eje vertical que pasa por el COM [kg·m²].

Ruedas y actuadores:

m_w: masa de cada rueda con su motor [kg].
I_w: inercia de cada rueda y motor alrededor de su eje de giro [kg·m²].
I'_w: inercia de cada rueda y motor alrededor de un diámetro [kg·m²].
N: relación de engranajes.
B_ω: fricción viscosa equivalente vista en el eje de rueda [N·m·s/rad].
τ_c: par de fricción de Coulomb.

Motores (eléctrico → mecánico):

Para convertir PWM a par.

K_t: constante de par [N·m/A].
K_e: constante de FEM [V·s/rad].
R_m, L_m: resistencia e inductancia del motor [Ω], [H].
V_{max} o rango de PWM.

Dimensiones y sensores:

b: distancia del arreglo de sensores al eje de ruedas [m] y su altura sobre suelo.

ancho del sensor array y **separación** entre sensores [m].

Con estos datos se pueden hacer los análisis cinemáticos y dinámicos del seguidor.

Cinemática (Sin fuerzas):

Con R y L mapeamos velocidades de rueda a (v, ω) y a $\dot{x}, \dot{y}, \dot{\theta}$:

$$v = \frac{R}{2}(\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{R}{2L}(\dot{\phi}_R - \dot{\phi}_L),$$

y el mapeo a coordenadas inerciales usando la orientación θ .

Dinámica (Con fuerzas):

Armamos el modelo tipo:

$$M(q) \ddot{q} + V(q, \dot{q}) \dot{q} = B(q) \tau - \Lambda(q)^T \lambda,$$

y luego eliminamos multiplicadores de Lagrange para obtener el modelo reducido en:

$$\eta = [\dot{\phi}_R, \dot{\phi}_L]^T$$

Allí entran $m_r, m_w, I_s, I_w, \dot{L}, d, R, L$ y los torques del motor.

Modelo motor y fricciones:

$$\tau_{R,L} = K_t i_{R,L} - \tau_{\text{fric}}$$

Con i derivada de V, K_e, R_m y la velocidad del eje.

Realizando la medición se tiene que:

Radio Rueda: $R = 0.02$ m.

Distancia entre ruedas: $2L = 0.14$ m $\Rightarrow L = 0.07$ m.

Distancia eje $\rightarrow$ QTR: $b = 0.15$ m.

Entonces la cinemática queda:

$$v = \frac{R}{2}(\dot{\phi}_R + \dot{\phi}_L) = 0.01(\dot{\phi}_R + \dot{\phi}_L)$$

$$\omega = \frac{R}{2L}(\dot{\phi}_R - \dot{\phi}_L) = \frac{0.02}{0.14}(\dot{\phi}_R - \dot{\phi}_L) \approx 0.142857(\dot{\phi}_R - \dot{\phi}_L)$$

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

Luego se usó Matlab para Calcular la masa total el offset “d” con la ubicación del centro de masa la inercia planar con ejes paralelos y la inercia de la rueda, además se definieron los pesos y posiciones de cada elemento ubicado en las 2 placas del seguidor de línea. Este código está como anexo en este documento posterior a las referencias.

```
Masa total m = 0.2840 kg
Offset COM d = 0.0240 m (hacia adelante desde eje)
Inercia planar I_s = 0.00037247 kg*m^2
Inercia rueda aprox I_w = 3.000000e-06 kg*m^2

M_eta =
    1.0e-04 *

    0.3900    0.2080
    0.2080    0.3900
```

Simulación hecha con $\tau_{R,L} = 0.030$ N*m, $\tau_{L,R} = 0.030$ N*m

Figura 7. Resultados iniciales en Matlab.

Estos valores siguiendo lo siguiente:

Variables:

ϕ_R, ϕ_L = posición angular ruedas derecha/izquierda (rad).

$\dot{\phi}_R, \dot{\phi}_L$ = velocidades angulares ruedas.

R = radio de rueda.

$2L$ = distancia entre ruedas (track), por lo tanto, L es la mitad.

m = masa total del robot (sin contar inercia de ruedas).

I_s = inercia planar del chasis alrededor del eje vertical que pasa por el COM.

I_w = inercia de cada rueda respecto a su eje.

Para cinemática:

$$v = \frac{R}{2}(\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{R}{2L}(\dot{\phi}_R - \dot{\phi}_L)$$

Siendo v la velocidad lineal del cuerpo y $\omega = \dot{\theta}$ la velocidad angular.

Se define la energía cinética total (traslación del chasis + rotación yaw + rotación ruedas):

$$T = \frac{1}{2}mv^2 + \frac{1}{2}I_s\omega^2 + \frac{1}{2}I_w(\dot{\phi}_R^2 + \dot{\phi}_L^2)$$

Donde se sustituyen v y ω y se organiza T en la forma:

$$T = \frac{1}{2} \begin{bmatrix} \dot{\phi}_R & \dot{\phi}_L \end{bmatrix} M_\eta \begin{bmatrix} \dot{\phi}_R \\ \dot{\phi}_L \end{bmatrix}$$

Con la matriz de masa reducida M_η :

$$M_\eta = \begin{bmatrix} ma^2 + I_s b^2 + I_w & ma^2 - I_s b^2 \\ ma^2 - I_s b^2 & ma^2 + I_s b^2 + I_w \end{bmatrix}$$

La ecuación dinámica a nivel de ruedas (si consideramos torques directos τ_R, τ_L aplicados sobre cada rueda y rozamientos viscosos B) sería:

$$M_\eta \ddot{\eta} + B_\eta \dot{\eta} = \tau, \quad \eta = \begin{bmatrix} \phi_R \\ \phi_L \end{bmatrix}, \quad \tau = \begin{bmatrix} \tau_R \\ \tau_L \end{bmatrix}$$

Donde B_η típicamente es diagonal con términos de fricción en los ejes de las ruedas (o suma de fricciones motor y rodadura).

Para el motor eléctrico se consideró:

$$\tau = K_t i, \quad V = R_m i + K_e \omega_m \quad (\text{o sea } i \approx (V - K_e \omega_m)/R_m)$$

Los parámetros de las masas de cada componente implementado se definieron en el código como se ve en la figura 8 a continuación, además se calculó la masa total (aproximadamente 300 gramos) y las inercias de las ruedas usando:

$$I_w \approx \frac{1}{2} m_{\text{rueda}} R^2$$

Dando un valor de $3 \times 10^{-6} \text{ kg} \cdot \text{m}^2$ para I_w y un valor de $3.7247 \times 10^{-4} \text{ kg} \cdot \text{m}^2$ para I_s .

```
% 1) Parámetros geométricos y estimaciones
R = 0.02; % m
L = 0.07; % m (mitad del track)
b = 0.15; % m (sensor)

% Masas (kg) - estimaciones iniciales
m_bat = 0.12;
m_L298 = 0.038;
m_caster = 0.007;
m_QTR = 0.005;
m_plate = 0.036;
m_esp32 = 0.008;
m_reg = 0.010;
m_whlR = 0.03;
m_whlL = 0.03;

% Posiciones x (m) desde eje de ruedas (+ hacia adelante)
x_bat = 0.01;
x_L298 = x_bat + 0.045;
x_caster = 0.14;
x_QTR = b;
x_plate = 0.05;
x_esp32 = 0.00;
x_reg = 0.00;
x_whlR = 0.00; % ruedas en el eje
x_whlL = 0.00;
```

Figura 8. Parámetros usados.

Una vez calculadas las inercias, la masa total y el offset de la distancia al centro de masa “d” se reemplaza en la matriz M_η :

$$M_\eta \approx \begin{bmatrix} 3.9002 \cdot 10^{-5} & 2.0798 \cdot 10^{-5} \\ 2.0798 \cdot 10^{-5} & 3.9002 \cdot 10^{-5} \end{bmatrix} \text{ kg m}^2$$

Y se graficó la velocidad lineal y angular de acuerdo con estos datos, como se ve en la figura 9:

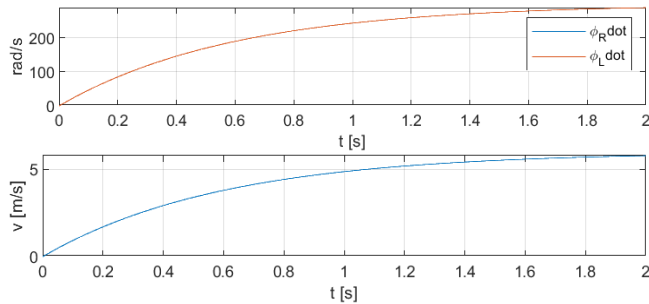


Figura 9. Velocidad angular y lineal respecto al tiempo.

Para los parámetros de los motores se estimaron estos valores a partir de pruebas con la esp 32 donde se examinaron sus rampas de comportamiento de voltaje y corriente y revisando sus hojas de datos.

$$\begin{aligned} K_t &= 0.03 \text{ Nm/A} \\ K_e &= 0.03 \text{ V s/rad} \\ R_m &= 2 \Omega \end{aligned}$$

Para los rozamientos viscosos se tomó $B_r = B_l = 1 \times 10^{-4} \text{ Nm s/rad}$.

La fricción eléctrica equivalente se determinó con $K_t * K_e / R_m$ Siendo 0.00045 Nm s-rad .

Ganancia torque vs voltaje: $G = \frac{K_t}{R_m} = \text{Análisis de resultados Nm/V}$.

A partir de la matriz M_η y la ecuación dinámica:

$$M_\eta \ddot{\eta} + B_{\text{tot}} \dot{\eta} = G V$$

Donde:

$$B_{\text{tot}} = B_\eta + D I_2$$

$$B_\eta = \text{diag}(B_r, B_l)$$

$$D = \frac{K_t K_e}{R_m}$$

$$G = \frac{K_t}{R_m}$$

Y para el espacio de estados se toman como estados, entradas y salidas:

$$z = \begin{bmatrix} \phi_R \\ \phi_L \\ \dot{\phi}_R \\ \dot{\phi}_L \end{bmatrix} \quad u = \begin{bmatrix} V_R \\ V_L \end{bmatrix} \quad y_{\text{dyn}} = \begin{bmatrix} \dot{\phi}_R \\ \dot{\phi}_L \end{bmatrix}$$

Donde:

$$\dot{z} = \begin{bmatrix} 0_2 & I_2 \\ 0_2 & -M_\eta^{-1} B_{\text{tot}} \end{bmatrix} z + \begin{bmatrix} 0_2 \\ M_\eta^{-1} G I_2 \end{bmatrix} u, \quad y_{\text{dyn}} = \begin{bmatrix} 0_{2 \times 2} & I_2 \end{bmatrix} z$$

Y reemplazando por los valores numéricos hallados se obtienen las matrices A, B, C y D.

$$A_{\text{dyn}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -19.70618065 & 10.50885624 \\ 0 & 0 & 10.50885624 & -19.70618065 \end{bmatrix}$$

$$B_{\text{dyn}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 537.44129046 & -286.60517006 \\ -286.60517006 & 537.44129046 \end{bmatrix}$$

$$C_{\text{dyn}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$D_{\text{dyn}} = 0$$

Modelo cinemático:

Modelo Dinámico (Geometría del movimiento):

Este se linealizó con matlab alrededor de $(\theta_0 = 0, \dot{\phi}_{RO} = \dot{\phi}_{LO} = 20 \text{ rad/s})$

$$\frac{x(s)}{\dot{\phi}_R} = \frac{0.01}{s}$$

$$\frac{x(s)}{\dot{\phi}_L} = \frac{0.01}{s}$$

Se definieron los estados y las entradas como:

$$s = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad u = \begin{bmatrix} \dot{\phi}_R \\ \dot{\phi}_L \end{bmatrix}$$

$$\frac{y(s)}{\dot{\phi}_R} = \frac{0.05714}{s^2}$$

$$\frac{y(s)}{\dot{\phi}_L} = \frac{-0.05714}{s^2}$$

Donde las ecuaciones son:

$$\frac{\theta(s)}{\dot{\phi}_R} = \frac{0.1429}{s}$$

$$\frac{\theta(s)}{\dot{\phi}_L} = \frac{-0.1429}{s}$$

$$\begin{aligned} v &= a(\dot{\phi}_R + \dot{\phi}_L) \\ \omega &= b(\dot{\phi}_R - \dot{\phi}_L) \\ \dot{x} &= v \cos \theta = a(\dot{\phi}_R + \dot{\phi}_L) \cos \theta \\ \dot{y} &= v \sin \theta = a(\dot{\phi}_R + \dot{\phi}_L) \sin \theta \\ \dot{\theta} &= \omega = b(\dot{\phi}_R - \dot{\phi}_L) \end{aligned}$$

Funciones de transferencia dinámico:

$$\frac{\dot{\phi}_R}{V_R} = \frac{537.4s + 7579}{s^2 + 39.41s + 277.9}$$

$$\frac{\dot{\phi}_L}{V_R} = \frac{-286.6s + 2.458 \times 10^{-6}}{s^2 + 39.41s + 277.9}$$

$$\frac{\dot{\phi}_R}{V_L} = \frac{-286.6s + 2.458 \times 10^{-6}}{s^2 + 39.41s + 277.9}$$

$$\frac{\dot{\phi}_L}{V_L} = \frac{537.4s + 7579}{s^2 + 39.41s + 277.9}$$

Jacobianos:

$$A_{\text{kin}}(s, u) = \frac{\partial f}{\partial s} = \begin{bmatrix} 0 & 0 & -a(\dot{\phi}_R + \dot{\phi}_L) \sin \theta \\ 0 & 0 & a(\dot{\phi}_R + \dot{\phi}_L) \cos \theta \\ 0 & 0 & 0 \end{bmatrix}$$

$$B_{\text{kin}}(s, u) = \frac{\partial f}{\partial u} = \begin{bmatrix} a \cos \theta & a \sin \theta \\ a \sin \theta & a \cos \theta \\ b & -b \end{bmatrix}$$

Como ya se tenía definido $R = 0.02$, $a = 0.01$, $L = 0.07\text{m}$, $b = 0.14285714$ se encontraron las matrices de estados.

$$A_{\text{kin}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0.4 \\ 0 & 0 & 0 \end{bmatrix}$$

$$B_{\text{kin}} = \begin{bmatrix} 0.01 & 0.01 \\ 0 & 0 \\ 0.14285714 & -0.14285714 \end{bmatrix}$$

$$C_{\text{kin}} = I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$D_{\text{kin}} = 0$$

Con las matrices de estados se pudo obtener las funciones de transferencia correspondientes con Matlab, el código se encuentra anexo posterior a las referencias.

Funciones de transferencia cinemático:

III. CONCLUSIONES

IV. REFERENCIAS

[1] R. Dhaouadi y A. Abu Hatab. "Dynamic Modelling of Differential-Drive Mobile Robots using Lagrange and Newton-Euler Methodologies: A Unified Framework". Hilaris Publishing SRL | Open Access JournalsHilaris Publishing SRL | Open Access Journals. Accedido el 15 de agosto de 2025. [En línea]. Disponible: <https://www.hilarispublisher.com/open-access/dynamic-modelling-of-differentialdrive-mobile-robots-using-lagrange-and-newtoneuler-methodologies-a-unified-framework-2168-9695.1000107.pdf>

Anexos:

Codigo para calcular inercias, masa y d:

```
%% modelo_seguidor.m
% Versión extendida: calcula COM, I_s, M_eta y simula
dinámica de ruedas -> (x,y,theta)
```

```
clear; close all; clc;
```

```
%% 1) Parámetros geométricos y estimaciones
```

```

R = 0.02;      % m
L = 0.07;      % m (mitad del track)
b = 0.15;      % m (sensor)

% Masas (kg) - estimaciones iniciales
m_bat = 0.12;
m_L298 = 0.038;
m_caster = 0.007;
m_QTR = 0.005;
m_plate = 0.036;
m_esp32 = 0.008;
m_reg = 0.010;
m_whR = 0.03;
m_whL = 0.03;

% Posiciones x (m) desde eje de ruedas (+ hacia adelante)
x_bat = 0.01;
x_L298 = x_bat + 0.045;
x_caster = 0.14;
x_QTR = b;
x_plate = 0.05;
x_esp32 = 0.00;
x_reg = 0.00;
x_whR = 0.00; % ruedas en el eje
x_whL = 0.00;

% Geometría de la placa para inercia aproximada
base_len = 0.14;
base_wid = 0.06;

% Inercia rueda (estimada como disco parcial)
m_rueda_est = 0.015;
I_w = 0.5 * m_rueda_est * R^2;

%% 2) Masa total y COM
masses_full = [m_bat m_L298 m_caster m_QTR m_plate
m_esp32 m_reg m_whR m_whL];
xs_full = [x_bat x_L298 x_caster x_QTR x_plate
x_esp32 x_reg x_whR x_whL];

m_total = sum(masses_full);
x_COM = sum(masses_full .* xs_full) / m_total; % offset
d

fprintf('Masa total m = %.4f kg\n', m_total);
fprintf('Offset COM d = %.4f m (hacia adelante desde
eje)\n', x_COM);

%% 3) Inercia planar I_s (aprox.)
I_plate_center = (1/12) * m_plate * (base_len^2 +
base_wid^2);
I_points = m_bat*(x_bat-x_COM)^2 + m_L298*(x_L298-
x_COM)^2 + ...
m_caster*(x_caster-x_COM)^2 +
m_QTR*(x_QTR-x_COM)^2 + ...
m_esp32*(x_esp32-x_COM)^2 + m_reg*(x_reg-
x_COM)^2 + ...
m_whR*(x_whR-x_COM)^2 + m_whL*(x_whL-
x_COM)^2;

I_plate = I_plate_center + m_plate*(x_plate-x_COM)^2;

I_s = I_points + I_plate;

fprintf('Inercia planar I_s = %.8f kg*m^2\n', I_s);
fprintf('Inercia rueda aprox I_w = %.6e kg*m^2\n\n', I_w);

%% 4) Matriz de masa reducida M_eta
a = R/2;
bcoeff = R/(2*L);

M11 = m_total * a^2 + I_s * bcoeff^2 + I_w;
M12 = m_total * a^2 - I_s * bcoeff^2;

M_eta = [M11, M12; M12, M11];

disp('M_eta = ');
disp(M_eta);

%% 5) Simulación simple: M_eta*phi_ddot + B*phi_dot
= tau
% Fricción viscosa estimada
B_r = 1e-4; % N*m*s/rad
B_l = 1e-4;
B_eta = diag([B_r B_l]);

% Ejemplo de torques aplicados (N*m) - paso en R y L
tau_R = 0.03; % N*m
tau_L = 0.03; % N*m
tau_func = @(t) [tau_R; tau_L];

% Sistema de estado: z = [phiR; phiL; phiRdot; phiLdot]
M_inv = inv(M_eta);

odefun = @(t,z) [...
z(3); ...
z(4); ...
M_inv*(tau_func(t) - B_eta*[z(3); z(4)])];

% condiciones iniciales (todo en cero)
z0 = zeros(4,1);

% tiempos
tspan = linspace(0,2,401);

% ode45 (convert to first order properly)
% We'll construct a wrapper for 4 states
odefun4 = @(t,z) [ z(3);
z(4);
M_inv*(tau_func(t) - B_eta*[z(3); z(4)]) ];

opts = odeset('RelTol',1e-6,'AbsTol',1e-8);
[tout,zout] = ode45(odefun4, tspan, z0, opts);

phiR = zout(:,1);
phiL = zout(:,2);
phiRdot = zout(:,3);
phiLdot = zout(:,4);

```



```
% Map to body velocities and pose
v = a*(phiRdot + phiLdot); % linear speed
omega = bcoeff*(phiRdot - phiLdot); % angular speed
```

```
% integrate pose
theta = cumtrapz(tout, omega);
x = cumtrapz(tout, v .* cos(theta));
y = cumtrapz(tout, v .* sin(theta));
```

```
% Plots
figure;
subplot(3,1,1); plot(tout, phiRdot, tout, phiLdot);
legend('phi_Rdot','phi_Ldot');
xlabel('t [s]'); ylabel('rad/s'); grid on;
subplot(3,1,2); plot(tout, v); xlabel('t [s]'); ylabel('v [m/s]');
grid on;
subplot(3,1,3); plot(x,y); axis equal; xlabel('x [m]');
ylabel('y [m]'); title('Trayectoria');
```

```
fprintf('\nSimulación hecha con tau_R=%.3f N*m,
tau_L=%.3f N*m\n',tau_R,tau_L);
```

Código para obtener las funciones de transferencia a partir de las matrices de estados

```
%% ft_cinematica_dinamica_named.m
% Funciones de transferencia con nombres claros de
entradas y salidas
```

```
clear; clc;
```

```
%% ===== CINEMÁTICO =====
% Estados: [x; y; theta]
% Entradas: [phiRdot; phiLdot]
% Salidas: [x; y; theta]
```

```
A_kin = [0 0 0;
0 0 0.4;
0 0 0];
```

```
B_kin = [0.01 0.01;
0 0;
0.14285714 -0.14285714];
```

```
C_kin = eye(3);
D_kin = zeros(3,2);
```

```
sys_kin = ss(A_kin,B_kin,C_kin,D_kin);
```

```
% Nombres de entradas y salidas
sys_kin.InputName = {'phiRdot','phiLdot'};
sys_kin.OutputName = {'x','y','theta'};
```

```
tf_kin = tf(sys_kin);
disp('==== Funciones de transferencia CINEMATICO
====');
tf_kin
```

```
%%=====DINÁMICO=====
% Estados: [phiR; phiL; phiRdot; phiLdot]
% Entradas: [VR; VL] (voltajes)
% Salidas: [phiRdot; phiLdot]
```

```
A_dyn = [0 0 1 0;
0 0 0 1;
0 0 -19.70618065 10.50885624;
0 0 10.50885624 -19.70618065];
```

```
B_dyn = [0 0;
0 0;
537.44129046 -286.60517006;
-286.60517006 537.44129046];
```

```
C_dyn = [0 0 1 0;
0 0 0 1];
D_dyn = zeros(2,2);
```

```
sys_dyn = ss(A_dyn,B_dyn,C_dyn,D_dyn);
```

```
% Nombres de entradas y salidas
sys_dyn.InputName = {'VR','VL'};
sys_dyn.OutputName = {'phiRdot','phiLdot'};
```

```
tf_dyn = tf(sys_dyn);
disp('==== Funciones de transferencia DINAMICO ===');
tf_dyn
```